

SMARTSAT RASTREADORES - Especificação técnica

Traccar 6.14, Live Video, ADAS/DSM, Evidências e Identificação de Motoristas

Use este arquivo como instrução principal para desenvolvimento no Codex.

Use também o AGENTS.md existente do projeto como regra base.

Atue como Engenheiro de Software Sênior, Arquiteto Frontend, Especialista Traccar e Orquestrador de Agentes.

Não faça perguntas desnecessárias. Analise o projeto, implemente as melhorias, teste, audite e entregue pronto para novo deploy no Railway.

0. Anexos, arquivos e links obrigatórios de referência

Todos os links e arquivos abaixo devem ser considerados parte da tarefa.

Nenhum recurso deve ser implementado por suposição quando houver documentação oficial ou arquivo técnico de referência.

0.1 Repositório base do frontend

<https://github.com/raphaeltoledo91/SMARTSAT-GPS>

Uso obrigatório:

1. Usar este repositório como base principal.
2. Não iniciar projeto do zero.
3. Preservar layout e funcionalidades existentes.
4. Criar branch nova antes de alterar código.
5. Subir alterações neste repositório se houver permissão.

0.2 Backend Traccar informado

Servidor Traccar padrão: <https://gps.smartsatrastreamento.com.br>
Versão informada: Traccar Server 6.14

URLs base configuráveis:

```
https://gps.smartsatrastreamento.com.br  
https://gps.smartsatrastreamento.com.br/api  
https://gps.smartsatrastreamento.com.br/api/stream
```

Variáveis esperadas:

```
TRACCAR_URL=https://gps.smartsatrastreamento.com.br  
TRACCAR_API_BASE_URL=https://gps.smartsatrastreamento.com.br/api  
TRACCAR_STREAM_BASE_URL=https://gps.smartsatrastreamento.com.br/api/stream
```

Regra:

1. Não usar IP hardcoded dentro de componentes.
2. A URL deve ser editável por `.env`, `data/config.local.json` ou runtime config segura.
3. Manter pronto para troca futura de backend.

0.3 Links oficiais Traccar

```
https://www.traccar.org/api-reference/  
https://www.traccar.org/api-reference/openapi.yaml  
https://raw.githubusercontent.com/traccar/traccar/master/openapi.yaml  
https://www.traccar.org/documentation/  
https://www.traccar.org/forums/  
https://www.traccar.org/traccar-api/
```

Uso obrigatório:

1. Validar endpoints no OpenAPI oficial antes de implementar.
2. Validar drivers, devices, positions, events, commands, permissions, reports, stream e websocket.
3. Não inventar endpoint.
4. Não inventar campos.
5. Não inventar comandos.
6. Registrar divergências em `docs/AUDIT_FINAL.md`.

0.4 Arquivos PDF técnicos enviados

Os PDFs abaixo devem ser anexados ou copiados para o repositório na pasta:

```
docs/references/
```

Arquivos obrigatórios:

```
docs/references/JTT 1078-2016 EN VERSION(1).pdf  
docs/references/WY02 3.16.pdf  
docs/references/WY02 USER MANUAL _ .pdf
```

Nomes originais recebidos:

```
JTT 1078-2016 EN VERSION(1).pdf
WY02 3.16.pdf
WY02 USER MANUAL _ .pdf
```

Se os arquivos não estiverem presentes no repositório local, criar docs/references/README.md informando:

```
Colocar nesta pasta os PDFs técnicos enviados:
- JTT 1078-2016 EN VERSION(1).pdf
url:(https://drive.google.com/file/d/18JX9Azw8tGxz3TFvIW1eF1BhtMk4aVq9/view?usp=sharing)
- WY02 3.16.pdf
url:(https://drive.google.com/file/d/1lkH0jLoL69_mq3feN027RmI0G6aHliRl/view?usp=sharing)
- WY02 USER MANUAL _ .pdf
url:(https://drive.google.com/file/d/19LDgUVvxtAx1qxwC7DiC6P-KuBAPAs_g/view?usp=sharing)
```

0.5 Uso técnico dos PDFs

JTT 1078-2016 EN VERSION(1).pdf

Usar para validar:

1. Comunicação de vídeo em veículos.
2. JT/T 1078.
3. Relação com JT/T 808.
4. Canal físico e canal lógico.
5. Áudio e vídeo em tempo real.
6. Stream principal/substream.
7. Playback remoto quando suportado.
8. Controle de reprodução.
9. Consulta de mídia histórica.
10. Limitações de protocolo.

Classificar implementação relacionada como:

```
NATIVO_TRACCAR
PARCIAL_TRACCAR
CUSTOMIZADO_PRODUTO
NAO_SUPORTADO
```

WY02 3.16.pdf

Usar para validar comandos textuais do equipamento WY02/H20P.

Comandos citados:

```
STATUSVIDEO
VERSION
SHUTDOWNTIME
TIMEZONE,A,B,C
TIMEZONE,AUTO
SHUTDOWNTIME,A
RATATION,A,B
DMS,REGION,A,B,C,D
REBOOT
RELAY,A
```

Validações obrigatórias:

```
RATATION canal: 1 ou 2
RATATION modo: 0, 1, 2 ou 3
DMS REGION coordenadas: 0 a 1000
RELAY: comando perigoso, exige permissão e confirmação
REBOOT: exige permissão administrativa operacional
TIMEZONE: validar formato
SHUTDOWNTIME: validar segundos
```

WY02 USER MANUAL□□□_□□□.pdf

Usar para validar:

1. Conformidade JT/T 808.
2. Conformidade JT/T 1078.
3. Suporte a vídeo em tempo real.
4. Reprodução quando suportada.
5. Duas câmeras.
6. Gravação 1080p/canal duplo.
7. ADAS.
8. DMS.
9. 4G.
10. GPS/BD.
11. Cartão TF.
12. Relé.
13. SOS.
14. Funções IA.
15. Comportamento do motorista.

0.6 Documentos que devem ser gerados no projeto

Criar ou atualizar:

```
docs/AUDIT_INITIAL.md
docs/AUDIT_FINAL.md
docs/ENVIRONMENT.md
```

```
docs/SECURITY.md
docs/DEPLOYMENT_RAILWAY.md
docs/DRIVER_IDENTIFICATION.md
docs/references/README.md
```

1. Repositório base obrigatório

Usar como base principal do frontend já criado:

```
https://github.com/raphaeltoledo91/SMARTSAT-GPS
```

Este repositório é a referência principal para evolução do projeto.

Não iniciar projeto do zero.

Antes de modificar:

1. Clonar ou abrir o repositório raphaeltoledo91/SMARTSAT-GPS.
2. Criar branch nova de trabalho.
3. Auditar estrutura atual.
4. Preservar layout, identidade visual e fluxo existente.
5. Manter compatibilidade com Vite/React, proxy Express, Railway, Docker e PM2 quando existirem.
6. Não remover funcionalidades existentes sem justificar na auditoria.
7. Não quebrar login, proxy, bootstrap, mapa, veículos, alertas, comandos, atributos, relatórios e telas já existentes.
8. Evoluir o frontend atual para suportar Traccar Server 6.14, live video, ADAS/DSM, evidências, permissões, whitelabel e identificação correta de motoristas.

Branch sugerida:

```
git checkout -b fix/traccar-driver-identification-live-video
```

Arquivos/pastas que devem ser analisados obrigatoriamente se existirem:

```
README.md
AGENTS.md
package.json
server.js
vite.config.js
railway.toml
Dockerfile
ecosystem.config.cjs
pm2.config.*
data/config.local.json
src/
docs/
public/
```

2. Objetivo principal

Modificar o projeto para produção avançada usando Traccar Server 6.14 como backend nativo.

Novo servidor Traccar padrão:

```
https://gps.smartsatrastreamento.com.br
```

Não deixar esse IP hardcoded dentro de componentes, hooks, services ou páginas.

Criar configuração editável para troca futura de backend.

Configuração mínima esperada:

```
TRACCAR_URL=https://gps.smartsatrastreamento.com.br
TRACCAR_API_BASE_URL=https://gps.smartsatrastreamento.com.br/api
TRACCAR_STREAM_BASE_URL=https://gps.smartsatrastreamento.com.br/api/stream

PUBLIC_APP_NAME=SMARTSAT
PUBLIC_CONTACT_PHONE=
PUBLIC_CONTACT_EMAIL=
PUBLIC_COMPANY_ADDRESS=
PUBLIC_PRIMARY_COLOR=
PUBLIC_SECONDARY_COLOR=
PUBLIC_LOGO_URL=

FEATURE_LIVE_VIDEO=true
FEATURE_ADAS_DSM_EVIDENCE=true
FEATURE_WORK_JOURNEY=true
FEATURE_WHITE_LABEL=true
FEATURE_RESELLER=true
FEATURE_DRIVER_IDENTIFICATION=true
```

Se o projeto usar frontend com variáveis públicas, expor somente o necessário.

Tokens, senhas, cookies, credenciais, sessão e segredos devem ficar protegidos no backend/proxy/BFF.

3. Fontes obrigatórias

Validar antes de implementar:

```
https://www.traccar.org/api-reference/
https://www.traccar.org/api-reference/openapi.yaml
https://raw.githubusercontent.com/traccar/traccar/master/openapi.yaml
https://www.traccar.org/documentation/
https://www.traccar.org/forums/
https://www.traccar.org/traccar-api/
```

Usar também os PDFs enviados como referência técnica:

```
docs/references/JTT 1078-2016 EN VERSION(1).pdf
docs/references/WY02 3.16.pdf
docs/references/WY02 USER MANUAL _ .pdf
```

Nunca inventar endpoint, campo, comando, permissão ou comportamento.

Classificar todos os recursos implementados como:

```
NATIVO_TRACCAR  
PARCIAL_TRACCAR  
CUSTOMIZADO_PRODUTO  
NAO_SUPORTADO
```

4. Regras de engenharia obrigatórias

Durante todo o desenvolvimento:

1. Validar e sanitizar entradas.
2. Validar limites de arrays, listas, datas, ids, canais e atributos.
3. Tratar erros sem expor stack trace ao usuário.
4. Proteger estado mutável e evitar race conditions.
5. Evitar duplicação de código.
6. Criar código modular, tipado e fácil de manter.
7. Não expor segredos no frontend.
8. Não fazer polling desnecessário.
9. Não sobrecarregar o backend Traccar.
10. Não alterar backend Traccar nativo.
11. Não quebrar funcionalidades existentes.
12. Não aplicar redesign completo sem necessidade.
13. Manter o layout atual.
14. Testar cada etapa.
15. Registrar tudo em auditoria.
16. Corrigir dados exibidos incorretamente no frontend.
17. Validar nomenclatura e campos conforme padrão Traccar.
18. Não confundir `Device.uniqueId` com `Driver.uniqueId`.
19. Não confundir motorista autorizado com motorista atual reportado na posição.

5. Fase 1 - Auditoria inicial obrigatória

Antes de editar código, criar:

docs/AUDIT_INITIAL.md

Conteúdo obrigatório:

Resumo do projeto
Stack detectada
Estrutura atual do projeto
Scripts disponíveis
Como o projeto roda localmente
Como o projeto faz build
Como o projeto faz deploy Railway
Pontos de integração Traccar existentes
Como autenticação está implementada
Como sessão/cookie/token são tratados
Como proxy Express/BFF está implementado
Como WebSocket/live data está implementado
Como comandos Traccar são enviados
Como eventos são consumidos
Como posições são consumidas
Como devices são exibidos
Como drivers são consumidos
Como driverUniqueId é tratado
Como a coluna Dispositivos > Motorista está implementada
Como motoristas são exibidos em mapa, popup, detalhes, relatórios e eventos
Como vídeo/live stream é tratado
Como atributos são tratados
Como permissões são tratadas
Se existe Redis/cache
Se existe rate limit
Riscos críticos
Erros de informação encontrados no frontend
Plano de implementação por etapas
Arquivos previstos para alteração

Não implementar nada antes desta auditoria.

6. Fase 2 - Auditoria crítica de identificação de motoristas

Esta fase é obrigatória porque já existe erro conhecido no frontend.

Erro encontrado:

Em Dispositivos, na coluna Motorista, não aparece corretamente a informação atribuída em driverUniqueId.

Objetivo:

1. Rever toda a estrutura de código que consome, normaliza, cruza e exibe informações de motoristas.
2. Corrigir campos errados.
3. Padronizar com a documentação/API do Traccar.
4. Garantir que driverUniqueId apareça corretamente quando existir.
5. Documentar todas as correções necessárias.

Pesquisar no código inteiro por:


```
driver
drivers
driverId
driverName
driverUniqueId
driverUniqueID
uniqueDriverId
uniqueId
attributes.driverUniqueId
position.driverUniqueId
position.attributes.driverUniqueId
device.driver
device.driverId
device.driverName
device.phone
device.contact
motorista
Motorista
```

Arquivos prováveis a revisar:

```
src/
src/components/
src/pages/
src/hooks/
src/services/
src/api/
src/utils/
src/lib/
src/config/
server.js
data/config.local.json
```

Se existirem, revisar especialmente:

```
DeviceTable
DevicesPage
DevicesList
VehicleList
VehicleCard
VehicleDetails
MapPopup
DevicePopup
DriverColumn
useDevices
useDrivers
usePositions
useBootstrap
traccarClient
traccarApi
normalizeDevice
normalizePosition
normalizeDriver
```

7. Padrão correto Traccar para motoristas

Implementar o frontend seguindo este contrato:

7.1 Entidade Driver

No Traccar, motorista deve ser tratado como entidade própria.

Modelo esperado:

```
type TraccarDriver = {  
  id: number;  
  name: string;  
  uniqueId: string;  
  attributes: Record<string, unknown>;  
};
```

Regras:

1. `Driver.uniqueId` é o identificador externo/único do motorista.
2. `Driver.name` é o nome humano do motorista.
3. `Driver.id` é o id interno do Traccar.
4. Não usar telefone do dispositivo como motorista.
5. Não usar contato do dispositivo como motorista.
6. Não usar `Device.uniqueId` como motorista.
7. Não usar `Device.phone` como motorista.
8. Não usar `Device.contact` como motorista.

7.2 Posição e motorista atual

O motorista atual normalmente deve vir da última posição reportada pelo dispositivo.

Modelo esperado:

```
type TraccarPosition = {  
  id: number;  
  deviceId: number;  
  protocol?: string;  
  serverTime?: string;  
  deviceTime?: string;  
  fixTime?: string;  
  valid?: boolean;  
  latitude?: number;  
  longitude?: number;  
  speed?: number;  
  attributes?: {  
    driverUniqueId?: string | number | null;  
    [key: string]: unknown;  
  };  
};
```

Regra principal:

A coluna Motorista em Dispositivos deve usar primeiro `position.attributes.driverUniqueId` da última posição válida do dispositivo.

7.3 Motorista autorizado não é necessariamente motorista atual

O vínculo via `/api/permissions` ou listagem via `/api/drivers?deviceId={deviceId}` pode representar motorista autorizado/vinculado ao dispositivo.

Não tratar esse vínculo como motorista atual se não houver `driverUniqueId` reportado na posição.

Regras:

1. `/api/drivers?deviceId={deviceId}` pode retornar motoristas vinculados/autorizados ao dispositivo.
2. Esse retorno não prova que o motorista está dirigindo naquele momento.
3. O motorista atual deve ser identificado por `position.attributes.driverUniqueId` quando o equipamento/protocolo reportar.
4. Se não houver `driverUniqueId`, exibir “Não informado” ou “Não reportado pelo dispositivo”.
5. Se houver motorista vinculado, exibir como “Autorizado” ou “Vinculado”, não como “Atual”, salvo regra documentada.

8. Correção obrigatória da coluna Dispositivos > Motorista

Corrigir a coluna Motorista na tabela/lista de Dispositivos.

Prioridade de resolução:

1. Última posição do dispositivo: `position.attributes.driverUniqueId`
2. Match com lista de drivers: `drivers.find(driver.uniqueId === driverUniqueId)`
3. Exibir `Driver.name + Driver.uniqueId` quando houver match
4. Exibir `driverUniqueId` bruto quando não houver match
5. Exibir motoristas vinculados somente como fallback informativo, com label "Autorizado/Vinculado"
6. Exibir "Não informado" quando não houver `driverUniqueId` nem vínculo

Implementar helper centralizado:

```
type DriverDisplaySource =
| 'position.attributes.driverUniqueId'
| 'drivers.uniqueId.match'
| 'drivers.deviceId.linked'
| 'none';

type DriverDisplay = {
label: string;
name?: string;
uniqueId?: string;
driverId?: number;
source: DriverDisplaySource;
isCurrent: boolean;
isLinkedOnly: boolean;
raw?: unknown;
};

function resolveDriverDisplay(params: {
deviceId: number;
position?: TraccarPosition | null;
drivers: TraccarDriver[];
linkedDrivers?: TraccarDriver[];
```

```
options?: {  
  showUniqueId?: boolean;  
  showName?: boolean;  
  fallbackToLinkedDriver?: boolean;  
};  
}): DriverDisplay;
```

Formato visual recomendado na coluna:

```
Se houver nome e uniqueId:  
João Silva · RFID123456  
  
Se houver apenas uniqueId:  
RFID123456  
  
Se houver apenas motorista vinculado/autorizado:  
João Silva · RFID123456 (Autorizado)  
  
Se não houver:  
Não informado
```

Nunca exibir:

```
telefone do dispositivo  
contato do dispositivo  
Device.uniqueId  
IMEI do rastreador  
id interno do device  
valor 0  
null  
undefined  
[object Object]
```

Tratar como valor inválido:

```
''  
'0'  
0  
'null'  
'undefined'  
'unknown'  
'UNKNOWN'  
'N/A'  
'_'
```

9. Correção de normalização de motoristas

Criar normalizadores centralizados:

```
normalizeDriver()  
normalizeDrivers()  
normalizeDriverUniqueId()  
normalizePositionDriverUniqueId()  
resolveDriverByUniqueId()  
resolveLinkedDriversByDeviceId()  
resolveCurrentDriverForDevice()  
normalizeDriverDisplay()
```

Implementação esperada:

```
function normalizeDriverUniqueId(value: unknown): string | null {
  if (value === null || value === undefined) return null;

  const normalized = String(value).trim();

  if (!normalized) return null;

  const invalidValues = new Set([
    '0',
    'null',
    'undefined',
    'unknown',
    'UNKNOWN',
    'N/A',
    '-',
  ]);

  if (invalidValues.has(normalized)) return null;

  return normalized;
}
```

Regras:

1. Comparar `Driver.uniqueId` e `position.attributes.driverUniqueId` como string normalizada.
 2. Não fazer comparação numérica pura, pois RFID/iButton pode ter zeros à esquerda.
 3. Preservar zeros à esquerda.
 4. Preservar letras quando existirem.
 5. Tratar `driverUniqueId` vindo como número convertendo para string sem perder compatibilidade.
 6. Não alterar dado original salvo se necessário para exibição.
 7. Não quebrar se `attributes` vier vazio.
-

10. Contrato de dados para bootstrap

Rever o bootstrap atual do frontend.

O bootstrap deve carregar ou disponibilizar:

```
devices
positions
drivers
events
groups
geofences
commands
server
user
permissions
```

Para motoristas, garantir:

1. Buscar `/api/drivers` quando usuário tiver permissão.

2. Buscar `/api/drivers?deviceId={deviceId}` somente quando necessário para vínculo por dispositivo.
3. Não fazer N+1 sem cache.
4. Não buscar `/api/drivers?deviceId=` para centenas de dispositivos em loop sem controle.
5. Criar cache por `deviceId` com TTL curto se precisar de motoristas vinculados.
6. Preferir resolver motorista atual por última posição.
7. Atualizar resolução de motorista ao receber nova posição no WebSocket.

Estrutura em memória recomendada:

```
type DriverIndexes = {  
  byId: Map<number, TraccarDriver>;  
  byUniqueId: Map<string, TraccarDriver>;  
  linkedByDeviceId: Map<number, TraccarDriver[]>;  
};
```

Criar índices uma vez por atualização:

```
driversById  
driversByUniqueId  
positionsByDeviceId  
linkedDriversByDeviceId
```

Não recalculer em cada renderização de linha da tabela sem memoização.

11. Correção no WebSocket

Quando chegar nova posição via `/api/socket`, atualizar motorista exibido.

Regras:

1. Receber `positions`.
2. Atualizar `positionsByDeviceId`.
3. Recalcular somente dispositivos afetados.
4. Se `position.attributes.driverUniqueId` mudou, atualizar coluna Motorista.
5. Se motorista saiu ou veio inválido, exibir "Não informado".
6. Não duplicar posição.
7. Não duplicar renderização.
8. Não fazer reload completo do bootstrap a cada mensagem.

Estados para diagnóstico:

```
type DriverDiagnostic = {  
  deviceId: number;  
  hasPosition: boolean;  
  hasAttributes: boolean;  
  hasDriverUniqueId: boolean;
```

```
driverUniqueId?: string;  
matchedDriver: boolean;  
matchedDriverName?: string;  
linkedDriversCount: number;  
source: DriverDisplaySource;  
};
```

12. Correção nas telas que exibem motorista

Rever e corrigir todas as telas que exibem motorista.

Locais obrigatórios:

```
Dispositivos  
Mapa  
Popup do mapa  
Detalhe do veículo  
Lista lateral de veículos  
Eventos  
Alertas  
Evidências  
ADAS/DSM  
Jornada de trabalho  
Relatórios  
Comandos quando relacionados a motorista  
Exportações CSV/PDF se existirem
```

Regra:

```
Todo lugar que exibir Motorista deve usar o mesmo helper resolveDriverDisplay().
```

Não criar lógica duplicada por tela.

13. Diagnóstico visual para motorista

Adicionar diagnóstico simples para operação, sem poluir o layout.

Em detalhe do veículo ou tooltip da coluna Motorista, exibir:

```
Fonte: Última posição  
driverUniqueId: RFID123456  
Motorista: João Silva  
Status: Identificado
```

Se não houver:

```
Fonte: Não reportado pelo equipamento  
driverUniqueId: ausente  
Motorista: Não informado  
Ação: Validar configuração do rastreador, iButton/RFID, protocolo ou atributo driverUniqueId.
```

Para motorista vinculado:

Fonte: Motorista vinculado/autorizado
Observação: Não representa necessariamente o motorista atual. O equipamento precisa reportar driverUniqueId na posição.

14. Correção de permissões para motoristas

Garantir permissões:

```
drivers.view  
drivers.manage  
devices.view  
positions.view  
events.view
```

Regras:

1. Usuário sem `drivers.view` pode ver `driverUniqueId` bruto se vier na posição e tiver acesso ao dispositivo.
2. Usuário sem `drivers.view` não deve receber dados completos de cadastro do motorista se a política do projeto bloquear.
3. Usuário sem permissão não deve conseguir editar motorista.
4. Vincular motorista a dispositivo deve usar permissão adequada.
5. Criar mensagem clara de acesso negado.

Permissões Traccar/entidade:

```
POST /api/permissions com deviceId e driverId cria vínculo entre dispositivo e motorista quando autorizado.  
DELETE /api/permissions remove vínculo.  
GET /api/drivers?deviceId={deviceId} lista motoristas vinculados/acessíveis ao dispositivo.
```

15. Correção de relatórios e jornada com motorista

Rever relatórios e jornada.

Regras:

1. Relatórios de viagem podem trazer `driverUniqueId` e `driverName`.
2. Não sobrescrever `driverName` do relatório com telefone.
3. Não sobrescrever `driverUniqueId` com `Device.uniqueId`.
4. Quando relatório vier sem motorista, tentar cruzar por posição somente se houver dados suficientes.

5. Documentar se o relatório do Traccar não retornou motorista.
6. Em jornada, usar driverUniqueId como chave externa quando existir.
7. Não inventar motorista por vínculo autorizado.

Modelo esperado:

```
type ReportDriverInfo = {  
  driverName?: string | null;  
  driverUniqueId?: string | null;  
  source: 'report' | 'position' | 'linked_driver' | 'none';  
};
```

16. Testes obrigatórios para identificação de motoristas

Criar testes unitários e/ou integração conforme stack atual.

Casos mínimos:

1. Dispositivo com posição contendo attributes.driverUniqueId e driver cadastrado com uniqueId igual deve exibir Driver.name + uniqueId.
2. Dispositivo com posição contendo attributes.driverUniqueId sem driver cadastrado deve exibir o driverUniqueId bruto.
3. Dispositivo sem position.attributes.driverUniqueId, mas com driver vinculado, deve exibir como Autorizado/Vinculado, não como Atual.
4. Dispositivo sem driverUniqueId e sem vínculo deve exibir Não informado.
5. Valor driverUniqueId = 0 deve ser tratado como não informado.
6. Valor driverUniqueId com zeros à esquerda deve ser preservado.
7. Device.phone não pode aparecer na coluna Motorista.
8. Device.uniqueId não pode aparecer na coluna Motorista.
9. Atualização de position via WebSocket deve atualizar motorista.
10. Remoção/ausência de driverUniqueId em nova posição deve atualizar para Não informado.
11. Usuário sem permissão de drivers não deve quebrar a tela.
12. Exportação deve usar o mesmo display padronizado.

Mock obrigatório:

```
const drivers = [  
  { id: 10, name: 'João Silva', uniqueId: '000123', attributes: {} },  
];  
  
const position = {  
  id: 99,  
  deviceId: 1,  
  attributes: {  
    driverUniqueId: '000123',  
  },  
};  
  
const device = {  
  id: 1,  
  name: 'Veículo 01',  
  uniqueId: 'IMEI999999',  
  phone: '+5511999999999',  
};
```

Resultado esperado na coluna Motorista:

João Silva · 000123

Resultado proibido:

+5511999999999
IMEI999999

17. Fase 3 - Configuração editável de backend

Implementar configuração centralizada para backend Traccar.

Criar ou ajustar conforme estrutura atual:

```
.env.example
.env.production.example
data/config.local.json
src/config/env.*
src/config/runtime.*
src/config/traccar.*
src/config/features.*
src/config/branding.*
```

Requisitos:

1. TRACCAR_URL editável.
2. TRACCAR_API_BASE_URL derivado ou configurável.
3. TRACCAR_STREAM_BASE_URL derivado ou configurável.
4. URL HTTPS padrão `https://gps.smartsatrastreamento.com.br`.
5. Fallback seguro quando variável estiver ausente.
6. Validação de URL.
7. Sanitização de valores vindos de JSON/env.
8. Compatibilidade com Railway.
9. Compatibilidade com execução local.
10. Compatibilidade com proxy Express existente.
11. Nenhum segredo exposto no browser.
12. Nenhum IP espalhado por componentes.

Configuração base esperada em JSON, se o projeto usar `data/config.local.json`:

```
{
  "traccarUrl": "https://gps.smartsatrastreamento.com.br",
  "traccarApiBaseUrl": "https://gps.smartsatrastreamento.com.br/api",
  "traccarStreamBaseUrl": "https://gps.smartsatrastreamento.com.br/api/stream",
  "port": 3000,
  "pollingMs": 30000,
  "authMode": "token-session-cookie",
  "allowUnsafeGoogleTiles": true,
```

```
"features": {  
  "liveVideo": true,  
  "adasDsmEvidence": true,  
  "workJourney": true,  
  "whiteLabel": true,  
  "reseller": true,  
  "driverIdentification": true  
}
```

Se o projeto já tiver chaves antigas, preservar compatibilidade.

18. Fase 4 - Whitelabel e customização da plataforma

Criar configuração editável para personalização básica da plataforma.

Campos obrigatórios:

```
type PlatformBrandingConfig = {  
  appName: string;  
  companyName?: string;  
  contactPhone?: string;  
  contactEmail?: string;  
  companyAddress?: string;  
  logoUrl?: string;  
  faviconUrl?: string;  
  primaryColor?: string;  
  secondaryColor?: string;  
  sidebarColor?: string;  
  loginBackgroundUrl?: string;  
};
```

Requisitos:

1. Nome da plataforma editável.
2. Logo editável.
3. Telefone editável.
4. E-mail editável.
5. Endereço editável.
6. Cor primária editável.
7. Cor secundária editável.
8. Favicon editável.
9. Fallback visual seguro.
10. Não quebrar layout existente.
11. Não aplicar cores inválidas.
12. Validar URLs de logo/favicon.
13. Criar documentação em docs/ENVIRONMENT.md.

Criar menu/tela se fizer sentido:

Configurações da Plataforma
Personalização
Aparência
Contato
Servidor
Módulos
Permissões

19. Fase 5 - Feature flags

Criar camada de feature flags para módulos e submódulos.

Flags mínimas:

```
type FeatureFlags = {  
  fleet: boolean;  
  map: boolean;  
  devices: boolean;  
  drivers: boolean;  
  events: boolean;  
  reports: boolean;  
  commands: boolean;  
  liveVideo: boolean;  
  adasDsmEvidence: boolean;  
  workJourney: boolean;  
  whiteLabel: boolean;  
  reseller: boolean;  
  permissions: boolean;  
  audit: boolean;  
  driverIdentification: boolean;  
};
```

Requisitos:

1. Feature flag deve controlar menu.
2. Feature flag deve controlar rota.
3. Feature flag deve controlar componentes.
4. Feature flag não substitui permissão.
5. Feature flag ausente deve usar valor seguro.
6. Registrar flags no docs/ENVIRONMENT.md.

20. Fase 6 - Contrato Traccar 6.14

Criar ou revisar cliente Traccar tipado:

```
traccarClient  
traccarApi  
traccarSocket  
traccarStream  
traccarCommands
```

```
traccarEvents
traccarDevices
traccarDrivers
traccarReports
traccarAttributes
```

Validar com OpenAPI oficial:

```
Session
Devices
Positions
Events
Drivers
Groups
Geofences
Maintenance
Reports
Commands
Attributes
Permissions
Stream
```

Regras:

1. Usar REST para bootstrap, CRUD, relatórios e comandos.
2. Usar WebSocket /api/socket para dados ao vivo quando suportado.
3. Usar HLS para vídeo ao vivo quando suportado.
4. Aplicar timeout em todas as chamadas.
5. Aplicar AbortController no frontend quando aplicável.
6. Tratar 400, 401, 403, 404, 409, 422 e 500.
7. Tratar CORS via proxy/BFF quando necessário.
8. Não expor stack trace.
9. Sanitizar todos os parâmetros.
10. Validar deviceId, groupId, driverId, eventId, positionId, channel.
11. Limitar intervalo máximo de relatórios.
12. Pagar tabelas e consultas grandes.
13. Evitar múltiplas conexões WebSocket por usuário.
14. Criar reconexão com backoff.
15. Encerrar conexões ao desmontar componentes.

21. Fase 7 - Proxy/BFF seguro

Validar o server.js atual.

Requisitos:

1. Manter credenciais fora do browser.

2. Encaminhar chamadas para Traccar via proxy seguro quando necessário.
3. Preservar cookies/sessão corretamente.
4. Aplicar CORS restrito por ambiente.
5. Aplicar rate limit em endpoints sensíveis.
6. Aplicar timeout.
7. Aplicar validação de URL.
8. Bloquear SSRF.
9. Não permitir proxy aberto para qualquer URL.
10. Não logar senha, token, cookie ou sessão.
11. Retornar erro amigável e seguro.
12. Documentar variáveis no `.env.example`.

Endpoints proxy sugeridos, se compatíveis com arquitetura atual:

```
/api/traccar/session  
/api/traccar/devices  
/api/traccar/positions  
/api/traccar/events  
/api/traccar/drivers  
/api/traccar/permissions  
/api/traccar/commands  
/api/traccar/stream  
/api/config/runtime
```

Nunca criar endpoint inseguro que aceite URL completa arbitrária.

22. Fase 8 - WebSocket e tempo real

Validar e implementar uso correto do WebSocket Traccar.

Requisitos:

1. Usar `/api/socket` quando sessão/cookie estiver válida.
2. Reaproveitar uma conexão por usuário quando possível.
3. Atualizar devices, positions e events em tempo real.
4. Atualizar motorista atual quando nova posição trazer `attributes.driverUniqueId`.
5. Ter fallback controlado para polling somente se WebSocket falhar.
6. Polling mínimo e configurável.
7. Backoff exponencial em reconexão.
8. Indicador visual de conexão.
9. Não duplicar eventos.
10. Não vaziar listeners.

11. Encerrar socket ao logout.

Estados mínimos:

```
connecting
connected
reconnecting
disconnected
unauthorized
error
```

23. Fase 9 - Live video por câmera/canal

Implementar módulo de vídeo mantendo o layout atual.

Módulos esperados:

```
VideoModule
LiveVideoPage
LiveVideoPanel
CameraChannelSelector
HlsVideoPlayer
VideoStreamStatus
VideoCommandControls
VehicleVideoIndicator
```

Fluxo esperado:

1. Usuário seleciona veículo.
2. Sistema valida permissão sobre deviceId.
3. Sistema lista canais disponíveis ou configurados.
4. Usuário escolhe canal.
5. Sistema valida se o dispositivo suporta vídeo.
6. Sistema valida comandos suportados pelo Traccar.
7. Sistema envia comando videoStart ou comando equivalente quando suportado.
8. Sistema abre HLS.
9. Player trata carregamento, timeout, erro de canal, stream vazio e encerramento.
10. Ao sair da tela ou trocar canal, enviar videoStop ou comando equivalente quando aplicável.
11. Não iniciar múltiplos streams duplicados sem necessidade.
12. Não sobrecarregar backend.
13. Exibir erro claro quando equipamento/protocolo não suportar vídeo.

Endpoints/fluxos a validar no código Traccar/OpenAPI antes de usar:

```
POST /api/commands/send
GET /api/commands/types?deviceId={deviceId}
GET /api/commands/send?deviceId={deviceId}
```

```
GET /api/stream/{deviceId}/{channel}/live.m3u8
GET /api/stream/{deviceId}/live.m3u8
```

Prioridade para Traccar 6.14:

```
/api/stream/{deviceId}/{channel}/live.m3u8
```

Fallback somente se validado:

```
/api/stream/{deviceId}/live.m3u8
```

Implementar com hls.js quando necessário.

Usar <video> HTML5 como base.

Estados obrigatórios do player:

```
idle
starting
waiting_stream
playing
stopping
offline
unsupported
channel_unavailable
timeout
unauthorized
error
```

Canais mínimos esperados:

```
type CameraChannel = {
  id: number;
  label: string;
  kind: 'front' | 'cabin' | 'rear' | 'side' | 'unknown';
  enabled: boolean;
  available?: boolean;
  streamUrl?: string;
};
```

Canais iniciais para WY02/H20P:

```
CH1 = câmera frontal
CH2 = câmera cabine
```

Não assumir que todos os dispositivos têm CH1/CH2.

Validar por atributos, comandos suportados ou configuração do dispositivo.

24. Fase 10 - Evidências ADAS/DSM

Criar módulo de evidências sem fingir que tudo é nativo do Traccar.

Estrutura esperada:

```
EvidenceModule
EvidenceList
EvidenceDetail
EvidenceTimeline
EvidenceFilters
EvidenceFromEventAction
EvidenceMediaViewer
EvidenceReviewPanel
EvidenceStatusBadge
EvidenceSeverityBadge
```

Modelo mínimo:

```
type EvidenceType =
| 'adas'
| 'dsm'
| 'speeding'
| 'harshAcceleration'
| 'harshBraking'
| 'harshCornering'
| 'fatigue'
| 'distraction'
| 'phoneUsage'
| 'smoking'
| 'laneDeparture'
| 'forwardCollision'
| 'pedestrianCollision'
| 'cameraBlocked'
| 'seatbelt'
| 'custom';

type EvidenceStatus =
| 'pending'
| 'reviewed'
| 'confirmed'
| 'rejected'
| 'archived';

type EvidenceSeverity =
| 'low'
| 'medium'
| 'high'
| 'critical';

type VideoEvidence = {
id: string;
tenantId?: string;
deviceId: number;
deviceName?: string;
driverId?: number;
driverUniqueId?: string;
driverName?: string;
eventId?: number;
positionId?: number;
channel?: number;
type: EvidenceType;
status: EvidenceStatus;
eventTime: string;
createdAt: string;
reviewedAt?: string;
reviewedBy?: string;
latitude?: number;
longitude?: number;
speed?: number;
severity?: EvidenceSeverity;
mediaUrl?: string;
```

```

snapshotUrl?: string;
streamUrl?: string;
source: 'traccar_event' | 'manual' | 'device_alarm' | 'custom_ai';
attributes: Record<string, unknown>;
};

```

Regras:

1. Eventos e posições vêm do Traccar nativo quando disponíveis.
2. Evidência persistida, revisão humana, mídia, snapshots, clips, score e analytics são camada customizada.
3. Playback histórico por SD card não deve ser prometido como nativo sem implementação validada.
4. Toda evidência deve vincular deviceId, eventId, positionId, driverUniqueId, channel e timestamp quando disponíveis.
5. Criar filtros por veículo, motorista, data, tipo, severidade e status.
6. Criar ação “Criar evidência a partir do evento”.
7. Criar viewer de mídia e fallback para ausência de mídia.
8. Criar auditoria de revisão.
9. Não quebrar quando não houver mídia.
10. Não quebrar quando não houver motorista.
11. Não quebrar quando atributo não existir.
12. Não duplicar evidência para o mesmo evento sem confirmação.
13. Usar o mesmo helper resolveDriverDisplay() para exibir motorista da evidência.

25. Fase 11 - ADAS/DSM e atributos

Criar mapeamento de atributos para eventos e posições.

Atributos esperados em camada de domínio:

```

type SafetyAttributes = {
  adas?: boolean;
  dsm?: boolean;
  fatigue?: boolean;
  distraction?: boolean;
  phoneUsage?: boolean;
  smoking?: boolean;
  laneDeparture?: boolean;
  forwardCollision?: boolean;
  headwayMonitoring?: boolean;
  pedestrianCollision?: boolean;
  yawning?: boolean;
  eyesClosed?: boolean;
  seatbelt?: boolean;
  cameraBlocked?: boolean;
  cameraChannel?: number;
  evidenceId?: string;
  videoAvailable?: boolean;
  driverUniqueId?: string;
};

```

Criar normalizadores:

```
normalizeTraccarEventToEvidence()  
normalizePositionAttributes()  
normalizeDeviceVideoCapabilities()  
normalizeDriverDisplay()  
normalizeSafetyAttributes()  
normalizeCameraChannels()
```

Motorista:

1. Exibir driverUniqueId quando disponível.
2. Não exibir telefone como motorista se existir driverUniqueId.
3. Buscar entidade Driver quando possível.
4. Tratar ausência de driver sem quebrar tela.
5. Documentar fallback no docs/AUDIT_FINAL.md.

Eventos ADAS/DSM devem ser mapeados por atributos quando disponíveis.

Exemplos de categorias:

```
Fadiga  
Distração  
Uso de telefone  
Fumo  
Olhos fechados  
Bocejo  
Saída de faixa  
Colisão frontal  
Pedestre  
Câmera bloqueada  
Cinto  
Excesso de velocidade  
Frenagem brusca  
Aceleração brusca  
Curva brusca
```

26. Fase 12 - Comandos e protocolos

Implementar camada segura de comandos.

Usar Traccar Commands nativo quando possível:

```
GET /api/commands/types?deviceId={deviceId}  
GET /api/commands/send?deviceId={deviceId}  
POST /api/commands/send  
GET /api/commands  
POST /api/commands  
PUT /api/commands/{id}  
DELETE /api/commands/{id}
```

Não enviar comando não suportado sem validação.

Criar catálogo de comandos por categoria:

```
type DeviceCommandCatalogItem = {
  key: string;
  label: string;
  protocolGroup:
  | 'jt808_jt1078'
  | 'teltonika'
  | 'suntech'
  | 'gt06'
  | 'concox'
  | 'jimy'
  | 'coban'
  | 'generic';
  traccarType?: string;
  textCommand?: string;
  requiresConfirmation: boolean;
  dangerous: boolean;
  permissions: string[];
  attributesSchema: Record<string, unknown>;
};
```

Comandos WY02/H20P a cadastrar como comandos textuais quando suportado:

```
STATUSVIDEO
VERSION
TIMEZONE,A,B,C
TIMEZONE,AUTO
SHUTDOWNTIME,A
RATATION,A,B
DMS,REGION,A,B,C,D
REBOOT
RELAY,A
```

Regras de segurança:

1. RELAY é perigoso.
2. RELAY exige permissão alta.
3. RELAY exige confirmação explícita.
4. RELAY exige auditoria.
5. REBOOT exige permissão administrativa operacional.
6. DMS,REGION deve validar coordenadas entre 0 e 1000.
7. RATATION deve validar canal 1 ou 2 e modo 0 a 3.
8. SHUTDOWNTIME deve validar segundos dentro de limite seguro.
9. TIMEZONE deve validar direção, horas e minutos.
10. Nunca permitir texto livre sem sanitização.
11. Logar comando sem expor credenciais.
12. Exibir resposta do comando de forma segura.
13. Bloquear comandos quando dispositivo estiver offline, salvo se Traccar permitir fila.

JT808/JT1078:

1. Validar suporte do device via Traccar.

2. Live video deve usar comando Traccar compatível, não raw binary no frontend.
3. Mapear canal físico/lógico.
4. Mapear stream principal/substream quando disponível.
5. Tratar canal indisponível.
6. Tratar timeout de segmentos HLS.
7. Tratar ausência de áudio.
8. Tratar câmera frontal/cabine como canais independentes.
9. Classificar funcionalidades como NATIVO, PARCIAL, CUSTOM ou NÃO SUPORTADO.

Teltonika, Suntech, GT06, Concox, Jimy, Coban:

1. Telemetria e comandos dependem do protocolo suportado pelo Traccar.
2. Não prometer live video para todos.
3. Se câmera/dashcam usar RTMP, vendor cloud ou protocolo não JT1078, criar adapter customizado ou MediaMTX separado.
4. Classificar recurso de vídeo como nativo, parcial ou customizado por protocolo/modelo.
5. Sempre mostrar suporte real por dispositivo.
6. Nunca mostrar botão de vídeo como funcional quando não houver suporte.

27. Fase 13 - Permissões por módulos e submódulos

Criar RBAC configurável.

Permissões mínimas:

```
fleet.view
fleet.manage

map.view

devices.view
devices.manage

drivers.view
drivers.manage

positions.view

events.view
events.manage

video.view
video.live
video.command.start
video.command.stop
video.channels.manage

evidence.view
evidence.create
evidence.review
evidence.delete
```

```
adas.view
dsm.view

commands.view
commands.send
commands.dangerous

journey.view
journey.manage

reports.view
reports.export

branding.view
branding.manage

tenant.view
tenant.manage

reseller.view
reseller.manage

settings.view
settings.manage

audit.view
```

Regras:

1. Proteger rotas.
2. Proteger menus.
3. Proteger botões.
4. Proteger chamadas API.
5. Validar no BFF quando existir.
6. Nunca depender somente de ocultar botão no frontend.
7. Criar fallback para usuário sem permissões.
8. Criar mensagem de acesso negado.
9. Registrar tentativa de comando perigoso.
10. Documentar permissões no docs/SECURITY.md.

28. Fase 14 - Jornada de trabalho

Se já existir módulo de jornada, revisar e manter.

Se não existir, preparar estrutura sem quebrar o projeto.

Módulos esperados:

```
WorkJourneyModule
DriverJourneyList
DriverJourneyDetail
JourneyStatusBadge
JourneyFilters
```

Campos mínimos:

```
type WorkJourneyStatus =  
| 'not_started'  
| 'working'  
| 'paused'  
| 'finished'  
| 'exceeded'  
| 'unknown';  
  
type DriverWorkJourney = {  
  id: string;  
  driverId?: number;  
  driverUniqueId?: string;  
  driverName?: string;  
  deviceId?: number;  
  startTime?: string;  
  endTime?: string;  
  drivingTimeMinutes?: number;  
  stoppedTimeMinutes?: number;  
  status: WorkJourneyStatus;  
  source: 'traccar' | 'custom' | 'manual';  
  attributes: Record<string, unknown>;  
};
```

Regras:

1. Não inventar dados.
2. Usar eventos, posições, ignição, movimento e motorista quando disponíveis.
3. Classificar o que é nativo e o que é cálculo customizado.
4. Permitir futura integração com legislação/regra operacional.
5. Não bloquear módulos principais se jornada estiver sem dados.
6. Usar driverUniqueId como chave externa quando disponível.
7. Não usar telefone como motorista.

29. Fase 15 - Desempenho e sobrecarga

Auditar e corrigir:

1. Remover polling desnecessário.
2. Usar WebSocket para tempo real.
3. Usar cache curto para snapshots e dashboards.
4. Não cachear posição em tempo real por tempo excessivo.
5. Aplicar debounce em filtros.
6. Aplicar paginação em tabelas.
7. Limitar datas em relatórios.
8. Usar lazy loading para módulos pesados.

9. Carregar player HLS apenas quando necessário.
10. Encerrar stream ao sair da tela.
11. Evitar múltiplas conexões WebSocket por usuário.
12. Reutilizar stream por device/canal quando possível.
13. Aplicar rate limit no BFF.
14. Aplicar timeout e abort controller.
15. Medir bundle e corrigir imports pesados.
16. Não carregar mapas, player e relatórios pesados ao mesmo tempo sem necessidade.
17. Evitar renderização de listas grandes sem paginação/virtualização.
18. Indexar drivers por uniqueId para evitar busca linear em cada linha.
19. Não fazer N+1 em `/api/drivers?deviceId`.

Se Redis existir ou for necessário:

1. Usar para sessão, cache curto e rate limit.
2. Não armazenar segredo em chave exposta.
3. Definir prefixo por ambiente.
4. Definir TTL explícito.
5. Criar fallback seguro se Redis estiver indisponível.
6. Documentar em `docs/ENVIRONMENT.md`.

Variáveis sugeridas:

```
REDIS_URL=  
REDIS_PREFIX=smartsat:  
SESSION_TTL_MS=86400000  
SNAPSHOT_CACHE_TTL_MS=5000  
EVENT_LOOKBACK_HOURS=24  
POLLING_MS=30000  
DRIVERS_CACHE_TTL_MS=30000
```

30. Fase 16 - Interface e layout

Manter estilo visual atual.

Adicionar sem quebrar layout:

1. Menu “Vídeo ao Vivo”.
2. Menu “Evidências”.
3. Menu “ADAS/DSM”.
4. Menu “Configurações da Plataforma”.
5. Menu “Permissões”.

6. Indicador de vídeo disponível no card do veículo.
7. Ação “Live Video” no detalhe do veículo.
8. Ação “Criar Evidência” em evento.
9. Visualização de canais CH1/CH2.
10. Estados vazios e mensagens claras.
11. Badges de severidade.
12. Badges de status.
13. Aviso quando equipamento não suportar vídeo.
14. Aviso quando servidor/stream estiver indisponível.
15. Aviso quando usuário não tiver permissão.
16. Coluna “Motorista” corrigida em Dispositivos.
17. Tooltip/diagnóstico de fonte do motorista.
18. Não exibir telefone como motorista.
19. Não exibir IMEI como motorista.

Não redesenhar a plataforma inteira sem necessidade.

31. Fase 17 - Módulos e submódulos esperados

Estrutura funcional esperada:

```
Dashboard
Mapa
Frota
Veículos
Dispositivos
Motoristas
Eventos
Alertas
Comandos
Vídeo ao Vivo
Evidências
ADAS/DSM
Jornada de Trabalho
Relatórios
Configurações
Whitelabel
Permissões
Auditoria
```

Submódulos de motoristas:

```
Identificação atual
Motoristas cadastrados
Motoristas vinculados/autorizados
driverUniqueId por posição
Validação de attributes.driverUniqueId
Diagnóstico por veículo
Correção da coluna Motorista
```

Submódulos de vídeo:

```
Canais
Player ao vivo
Status do stream
Comando iniciar stream
Comando parar stream
Histórico de erros
Capacidade do dispositivo
```

Submódulos de evidência:

```
Lista de evidências
Filtros
Detalhe
Timeline
Mídia
Snapshot
Revisão
Auditoria
Exportação futura
```

Submódulos de configurações:

```
Servidor Traccar
Branding
Módulos
Permissões
Contato
Railway/env
```

32. Fase 18 - Produção Railway

Preparar para deploy futuro no Railway, mas não fazer deploy agora.

Criar ou atualizar:

```
README.md
docs/DEPLOYMENT_RAILWAY.md
docs/ENVIRONMENT.md
docs/SECURITY.md
docs/DRIVER_IDENTIFICATION.md
docs/AUDIT_FINAL.md
docs/references/README.md
```

Documentar variáveis:

```
TRACCAR_URL
TRACCAR_API_BASE_URL
TRACCAR_STREAM_BASE_URL
PUBLIC_APP_URL
PUBLIC_APP_NAME
PUBLIC_CONTACT_PHONE
PUBLIC_CONTACT_EMAIL
PUBLIC_COMPANY_ADDRESS
PUBLIC_PRIMARY_COLOR
PUBLIC_SECONDARY_COLOR
```

```
PUBLIC_LOGO_URL
CORS_ORIGINS
COOKIE_SECURE
COOKIE_SAMESITE
REDIS_URL
REDIS_PREFIX
SESSION_TTL_MS
POLLING_MS
SNAPSHOT_CACHE_TTL_MS
EVENT_LOOKBACK_HOURS
DRIVERS_CACHE_TTL_MS
FEATURE_LIVE_VIDEO
FEATURE_ADAS_DSM_EVIDENCE
FEATURE_WORK_JOURNEY
FEATURE_WHITE_LABEL
FEATURE_RESELLER
FEATURE_DRIVER_IDENTIFICATION
```

Railway checklist:

```
Variáveis configuradas
Build local aprovado
Dockerfile validado
railway.toml validado
Porta configurável
Healthcheck existente ou documentado
Logs sem segredo
Servidor Traccar acessível
CORS validado
Cookies seguros em produção
Identificação de motoristas validada
```

Não executar deploy no Railway nesta tarefa.

Apenas deixar pronto e informar comandos.

33. Fase 19 - Testes obrigatórios

Executar conforme stack existente:

```
npm install
npm run lint
npm run typecheck
npm run test
npm run build
```

Se algum script não existir, documentar no docs/AUDIT_FINAL.md e criar quando fizer sentido.

Testes mínimos gerais:

1. Config carrega com IP padrão.
2. Config permite trocar servidor.
3. Config rejeita URL inválida.
4. Login/session não quebra.
5. REST Traccar trata erro.
6. WebSocket reconecta.
7. Devices renderizam com campos nulos.
8. Driver exibe driverUniqueId.
9. Driver não exibe telefone quando driverUniqueId existir.

10. Driver não exibe Device.uniqueId como motorista.
11. Live video inicia canal.
12. Player HLS trata erro.
13. Troca de canal encerra anterior.
14. Saída da tela encerra stream quando aplicável.
15. Evidência cria a partir de evento.
16. Evidência não quebra sem mídia.
17. RBAC bloqueia usuário sem permissão.
18. Comando perigoso exige confirmação.
19. Comando inválido é bloqueado.
20. Build final sem erro.
21. Railway config não quebra.

Testes mínimos específicos de motorista:

1. position.attributes.driverUniqueId com match em Driver.uniqueId exibe nome + uniqueId.
2. position.attributes.driverUniqueId sem match exibe uniqueId bruto.
3. Motorista vinculado sem driverUniqueId de posição aparece como Autorizado/Vinculado.
4. Sem driverUniqueId e sem vínculo aparece Não informado.
5. driverUniqueId = 0 não aparece como motorista válido.
6. driverUniqueId com zeros à esquerda preserva zeros.
7. device.phone nunca aparece na coluna Motorista.
8. device.contact nunca aparece na coluna Motorista.
9. device.uniqueId nunca aparece na coluna Motorista.
10. Atualização via WebSocket altera a coluna Motorista.
11. Remoção do driverUniqueId em posição nova altera para Não informado.
12. Exportação usa o mesmo valor da tela.

34. Fase 20 - Auditoria final obrigatória

Criar:

docs/AUDIT_FINAL.md

Conteúdo obrigatório:

Resumo executivo
 Repositório base usado
 Branch criada
 Commits realizados
 Arquivos alterados
 Arquivos PDF anexados em docs/references
 Links oficiais usados
 Funcionalidades implementadas
 Correção de identificação de motoristas
 Como Driver.uniqueId foi tratado
 Como position.attributes.driverUniqueId foi tratado
 Como /api/drivers?deviceId foi tratado
 Como motorista vinculado foi diferenciado de motorista atual
 Correção da coluna Dispositivos > Motorista
 Telas corrigidas que exibiam motorista
 Campos errados removidos da exibição
 O que é nativo Traccar
 O que é parcialmente nativo
 O que é customizado
 O que não é suportado ou depende do equipamento
 Configurações criadas
 Comandos criados
 Atributos criados
 Permissões criadas
 Correções de segurança
 Correções de desempenho

Correções de layout
Correções de backend/proxy
Riscos restantes
Testes executados
Resultado dos testes
Checklist para Railway
Próximos passos recomendados

A auditoria final deve ser objetiva e útil para produção.

35. Fase 21 - Git e entrega no repositório

Após passar testes:

1. Criar branch de trabalho.
2. Aplicar commits organizados.
3. Subir para o repositório remoto se houver permissão.
4. Não alterar backend Traccar nativo.
5. Não fazer deploy no Railway.
6. Informar comandos finais para eu refazer deploy.

Sequência sugerida:

```
git status
git checkout -b fix/traccar-driver-identification-live-video
npm install
npm run lint
npm run typecheck
npm run test
npm run build
git add .
git commit -m "docs: add Traccar reference links and device PDF references"
git commit -m "fix: correct Traccar driver identification and driverUniqueId display"
git commit -m "feat: add Traccar 6.14 live video, ADAS DSM evidence and configurable platform"
git push origin fix/traccar-driver-identification-live-video
```

Se já estiver em branch criada, não recriar.

36. Regras de compatibilidade Traccar

Não alterar o backend Traccar nativo.

Não criar dependência obrigatória de backend customizado se o recurso puder usar Traccar nativo.

Sempre respeitar:

Traccar REST API
Traccar WebSocket

```
Traccar Commands
Traccar Attributes
Traccar Events
Traccar Drivers
Traccar Permissions
Traccar Reports
Traccar Stream
```

Quando não for nativo, criar camada isolada e documentar como customizada.

37. Regras para live video JT808/JT1078

Vídeo ao vivo pode depender de:

```
Modelo do equipamento
Protocolo
Canal configurado
Comando aceito pelo Traccar
Suporte do Traccar 6.14
HLS disponível
Rede móvel do dispositivo
Servidor acessível
Portas liberadas
```

Não exibir como funcional quando não houver suporte.

Estados de suporte por dispositivo:

```
type VideoSupportStatus =
| 'supported'
| 'partially_supported'
| 'unsupported'
| 'unknown'
| 'offline'
| 'misconfigured';
```

Exibir mensagem clara:

```
Vídeo não disponível para este dispositivo.
Canal indisponível.
Dispositivo offline.
Comando de vídeo não suportado.
Stream ainda não gerado.
Servidor de stream indisponível.
Sem permissão para visualizar vídeo.
```

38. Regras para evidências com mídia

Não prometer:

```
download histórico automático
clip automático do SD card
```

```
playback remoto histórico  
IA no servidor  
snapshot automático  
reprodução de eventos antigos
```

sem validação técnica.

Implementar fallback:

```
Evidência sem mídia  
Evidência vinculada ao evento  
Evidência vinculada à posição  
Evidência com atributos  
Evidência com driverUniqueId quando disponível  
Evidência com canal provável  
Evidência com status pendente
```

39. Regras para dispositivos WY02/H20P

Considerar como referência:

```
Conformidade JT/T 808  
Conformidade JT/T 1078  
Duas câmeras  
Canal frontal  
Canal cabine  
ADAS  
DMS  
4G  
GPS/BD  
Cartão TF  
Vídeo em tempo real  
Reprodução quando suportada  
Alertas  
TTS  
Botão SOS  
Relé
```

Comandos textuais:

```
STATUSVIDEO  
VERSION  
SHUTDOWNTIME  
TIMEZONE,A,B,C  
TIMEZONE,AUTO  
SHUTDOWNTIME,A  
RATATION,A,B  
DMS,REGION,A,B,C,D  
REBOOT  
RELAY,A
```

Validações:

```
Canal: 1 ou 2  
RATATION modo: 0, 1, 2 ou 3  
DMS REGION coordenadas: 0 a 1000  
RELAY: ON/OFF ou equivalente validado  
SHUTDOWNTIME: número seguro de segundos
```

TIMEZONE: formato válido

40. Resultado final esperado

Mensagem final esperada do Codex:

```
Implementação concluída.

Repositório base:
https://github.com/raphaeltoledo91/SMARTSAT-GPS

Branch:
Commits:
Arquivos principais:
Arquivos PDF anexados:
Links oficiais usados:
Configuração Traccar aplicada:
Correção de identificação de motoristas:
Coluna Dispositivos > Motorista:
driverUniqueId:
Live video:
ADAS/DSM:
Evidências:
Whitelabel:
Permissões:
Correções de segurança:
Correções de desempenho:
Testes executados:
Status:
Pendências:
Comando sugerido para redeploy Railway:
```

41. Critério de aceite

A tarefa só é considerada concluída se:

1. O projeto compilar.
2. O build passar.
3. O IP padrão estiver configurável.
4. O backend Traccar não estiver hardcoded.
5. O layout atual for preservado.
6. A coluna Dispositivos > Motorista estiver corrigida.
7. driverUniqueId aparecer quando existir em position.attributes.driverUniqueId.
8. Driver.uniqueId for usado corretamente para cruzamento com o cadastro de motoristas.
9. Telefone não aparecer mais como motorista.
10. Device.uniqueId não aparecer mais como motorista.
11. Motorista vinculado for diferenciado de motorista atual.
12. O módulo de live video existir.

13. O módulo de evidências existir.
 14. ADAS/DSM estiver mapeado.
 15. Permissões forem aplicadas.
 16. Whitelabel estiver editável.
 17. Comandos perigosos forem protegidos.
 18. Auditoria inicial existir.
 19. Auditoria final existir.
 20. Documentação Railway existir.
 21. Documentação docs/DRIVER_IDENTIFICATION.md existir.
 22. Pasta docs/references/ existir.
 23. Todos os arquivos PDF de referência estiverem anexados ou listados em docs/references/README.md.
 24. Todos os links oficiais estiverem documentados.
 25. Repositório estiver pronto para novo deploy.
-

42. Observação final importante

Vídeo ao vivo pode ser nativo via Traccar 6.14, JT808/JT1078 e HLS quando o equipamento, comando, canal e servidor estiverem corretamente configurados.

Evidências com mídia, revisão humana, snapshots, clips, score, analytics, armazenamento customizado e playback histórico devem ser tratados como camada customizada do produto, salvo validação explícita no backend Traccar, no equipamento e na documentação oficial.

Identificação de motoristas deve seguir o padrão Traccar:

```
Cadastro do motorista: Driver.uniqueId
Motorista atual reportado: position.attributes.driverUniqueId
Cruzamento correto: Driver.uniqueId === position.attributes.driverUniqueId
```

Não implementar promessas falsas.

Não mascarar limitação técnica.

Não exibir dados errados.

Registrar tudo na auditoria final.